

# Alert Notification Basics

**Series:** WFLOW | **Notebook:** 3 of 9 | **Created:** January 2026 | **Last Updated:** 04/03/2026

## Sending Notifications with Workflows

The most common workflow use case is sending alert notifications to Slack, Microsoft Teams, and email. This notebook covers setting up connections, configuring notification tasks, and best practices for effective alerting.

## Table of Contents

- 1. [Notification Architecture](#)
- 2. [Setting Up Connections](#)
- 3. [Slack Notifications](#)
- 4. [Microsoft Teams Notifications](#)
- 5. [Email Notifications](#)
- 6. [Message Formatting](#)
- 7. [Complete Alert Workflow Example](#)

## Prerequisites

| Requirement           | Details                         |
|-----------------------|---------------------------------|
| Dynatrace Environment | SaaS with Platform subscription |

| Requirement     | Details                                                                                |
|-----------------|----------------------------------------------------------------------------------------|
| Permissions     | <code>automation:workflows:write</code> ,<br><code>automation:connections:write</code> |
| Prior Knowledge | <b>WFLOW-01</b> and <b>WFLOW-02</b>                                                    |
| External Access | Slack workspace admin or Teams channel access                                          |

# 1. Notification Architecture

## How Workflow Notifications Work

```

Trigger (Davis Problem, Schedule, etc.)
    ↓
Workflow Executes
    ↓
Notification Task
    ↓
Connection (credentials)
    ↓
External Service (Slack, Teams, Email)

```

## Components

| Component                | Purpose                                        |
|--------------------------|------------------------------------------------|
| <b>Connection</b>        | Securely stores credentials (tokens, webhooks) |
| <b>Notification Task</b> | Sends message using connection                 |
| <b>Message Template</b>  | Dynamic content using Jinja expressions        |

## Supported Notification Channels

| Channel         | Task Type                              | Authentication                         |
|-----------------|----------------------------------------|----------------------------------------|
| Slack           | <code>dynatrace.slack:message</code>   | OAuth App or Webhook                   |
| Microsoft Teams | <code>dynatrace.msTeams:message</code> | Power Automate or Webhook (deprecated) |

| Channel        | Task Type                           | Authentication      |
|----------------|-------------------------------------|---------------------|
| Email          | <code>dynatrace.email:send</code>   | SMTP or built-in    |
| PagerDuty      | <code>dynatrace.pagerduty:*</code>  | Integration Key     |
| ServiceNow     | <code>dynatrace.servicenow:*</code> | OAuth or Basic Auth |
| Jira           | <code>dynatrace.jira:*</code>       | API Token           |
| Custom Webhook | <code>dynatrace.http:request</code> | Bearer/Basic/Custom |

## 2. Setting Up Connections

Connections store credentials separately from workflows for security and reusability.

### Accessing Connections

1. Open **Settings** (gear icon)
2. Navigate to **Integration** → **Connections**
3. Or use direct URL: `https://<env>/ui/apps/dynatrace.hub/connections`

### Creating a Connection

1. Click **+ Connection**
2. Select connection type (Slack, Teams, etc.)
3. Enter credentials
4. Name the connection (e.g., `slack-production-alerts` )
5. Save

### Connection Best Practices

| Practice                      | Description                                            |
|-------------------------------|--------------------------------------------------------|
| <b>Descriptive names</b>      | <code>slack-prod-oncall</code> not <code>slack1</code> |
| <b>Environment separation</b> | Different connections for prod/staging                 |
| <b>Least privilege</b>        | Minimum required scopes                                |

| Practice         | Description                |
|------------------|----------------------------|
| Regular rotation | Update tokens periodically |

## 3. Slack Notifications

---

### Option 1: Slack App (Recommended)

Provides richer features: channels, DMs, reactions, threads.

**Setup:**

1. Create Slack App at <https://api.slack.com/apps>
2. Add OAuth scopes: `chat:write` , `chat:write.public`
3. Install to workspace
4. Copy Bot User OAuth Token
5. Create connection in Dynatrace with token

### Option 2: Incoming Webhook (Simpler)

Single channel, no advanced features.

**Setup:**

1. Go to Slack channel settings → Integrations
2. Add **Incoming Webhook**
3. Copy webhook URL
4. Create connection with webhook URL

## Basic Slack Message Task

```
name: send_slack_alert
type: dynatrace.slack:message
input:
  connection: slack-production
  channel: "#alerts-production"
  message: |
    :rotating_light: *Problem Detected*

    *Title:* {{ event()["title"] }}
    *Severity:* {{ event()["severity"] }}
    *Started:* {{ event()["start_time"] }}

    &lt;{{ event()["problem_url"] }}|View in Dynatrace&gt;
```

## Slack Message with Blocks

For richer formatting:

```

input:
  connection: slack-production
  channel: "#alerts"
  blocks:
    - type: header
      text:
        type: plain_text
        text: "{{ ':red_circle:' if event()['severity'] == 'CRITICAL' else
':large_orange_circle:' }} {{ event()['severity'] }} Alert"
    - type: section
      fields:
        - type: mrkdown
          text: "*Problem:*\\n{{ event()['title'] }}"
        - type: mrkdown
          text: "*Status:*\\n{{ event()['status'] }}"
    - type: section
      fields:
        - type: mrkdown
          text: "*Started:*\\n{{ event()['start_time'] }}"
        - type: mrkdown
          text: "*ID:*\\n{{ event()['display_id'] }}"
    - type: actions
      elements:
        - type: button
          text:
            type: plain_text
            text: "View Problem"
          url: "{{ event()['problem_url'] }}"

```

## 4. Microsoft Teams Notifications

**Warning — O365 Connectors Deprecated:** Microsoft retired Office 365 Connectors (Incoming Webhooks) in late 2024. Existing webhooks may continue to work temporarily, but **new O365 Connector creation is disabled**. Use one of these alternatives instead:

| Method                         | Status      | Notes                                                                                                                                           |
|--------------------------------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Power Automate Workflow</b> | Recommended | Create a "When a Teams webhook request is received" flow in Power Automate; use the resulting URL as a <code>dynatrace.http:request</code> task |

| Method                           | Status      | Notes                                                                 |
|----------------------------------|-------------|-----------------------------------------------------------------------|
| <b>Teams Workflows Connector</b> | Recommended | Available in new Teams client under channel ...<br>→ <b>Workflows</b> |
| <b>O365 Incoming Webhook</b>     | Deprecated  | Legacy method shown below for reference only                          |

## Setting Up Teams Webhook (Legacy)

1. Open target Teams channel
2. Click ... → **Connectors** (or **Workflows** in new Teams)
3. Add **Incoming Webhook**
4. Name it (e.g., "Dynatrace Alerts")
5. Copy the webhook URL
6. Create connection in Dynatrace

## Basic Teams Message Task

```
name: send_teams_alert
type: dynatrace.msteams:message
input:
  connection: teams-production
  message: |
    **Problem Detected**

    **Title:** {{ event()["title"] }}
    **Severity:** {{ event()["severity"] }}
    **Started:** {{ event()["start_time"] }}

    [View in Dynatrace]({{ event()["problem_url"] }})
```

## Teams Adaptive Card

For richer formatting:

```

input:
  connection: teams-production
  card:
    type: AdaptiveCard
    $schema: "http://adaptivecards.io/schemas/adaptive-card.json"
    version: "1.4"
    body:
      - type: TextBlock
        text: "{{ event()['severity'] }}" Alert"
        size: Large
        weight: Bolder
        color: "{{ 'Attention' if event()['severity'] == 'CRITICAL' else
'Warning' }}"
      - type: FactSet
        facts:
          - title: "Problem"
            value: "{{ event()['title'] }}"
          - title: "Status"
            value: "{{ event()['status'] }}"
          - title: "Started"
            value: "{{ event()['start_time'] }}"
    actions:
      - type: Action.OpenUrl
        title: "View in Dynatrace"
        url: "{{ event()['problem_url'] }}"

```

## 5. Email Notifications

### Email Options

| Option              | Setup                 | Best For                |
|---------------------|-----------------------|-------------------------|
| <b>Built-in</b>     | No setup required     | Simple notifications    |
| <b>Custom SMTP</b>  | Configure SMTP server | Corporate email servers |
| <b>SendGrid/SES</b> | API integration       | High volume, templates  |



## Basic Email Task

```
name: send_email_alert
type: dynatrace.email:send
input:
  to:
    - "oncall@company.com"
    - "platform-team@company.com"
  subject: "[{{ event()['severity'] }}] [{{ event()['title'] }}]"
  body: |
    A problem has been detected in your Dynatrace environment.

    Problem Details:
    -----
    Title: [{{ event()["title"] }}]
    Severity: [{{ event()["severity"] }}]
    Status: [{{ event()["status"] }}]
    Start Time: [{{ event()["start_time"] }}]
    Problem ID: [{{ event()["display_id"] }}]

    Affected Entities:
    [{{ event()["affected_entity_ids"] | join(", ") }}]

    Root Cause:
    [{{ event()["root_cause_entity_id"] }}]

    View this problem:
    [{{ event()["problem_url"] }}]
```

# HTML Email

```
input:
  to: ["oncall@company.com"]
  subject: "[{{ event()['severity'] }}] [{{ event()['title'] }}"
  contentType: "text/html"
  body: |
    <html>
    <body style="font-family: Arial, sans-serif;">
      <h2 style="color: {{ '#dc3545' if event()['severity'] == 'CRITICAL'
else '#ffc107' }};">
        {{ event()['severity'] }} Alert
      </h2>
      <table style="border-collapse: collapse;">
        <tr><td><b>Problem:</b></td><td>
          [{{ event()['title'] }}]</td></tr>
        <tr><td><b>Status:</b></td><td>[{{
event()['status'] }}]</td></tr>
        <tr><td><b>Started:</b></td><td>
          [{{ event()['start_time'] }}]</td></tr>
      </table>
      <p><a href="[{{ event()['problem_url'] }}">View in
Dynatrace</a></p>
    </body>
    </html>
```

## 6. Message Formatting

### Jinja Expression Reference

| Expression                                                     | Result                   | Use               |
|----------------------------------------------------------------|--------------------------|-------------------|
| <code>{{ event()["title"] }}</code>                            | Problem title            | Main content      |
| <code>{{ event()["severity"] }}</code>                         | CRITICAL/HIGH/MEDIUM/LOW | Color coding      |
| <code>{{ event()["affected_entity_ids"] \ join(", ") }}</code> | Comma-separated list     | Show all affected |
| <code>{{ event()["start_time"] }}</code>                       | ISO timestamp            | When started      |

| Expression                                | Result   | Use             |
|-------------------------------------------|----------|-----------------|
| <code>{{ event()["problem_url"] }}</code> | Full URL | Link to problem |

## Conditional Formatting

```
{% if event()["severity"] == "CRITICAL" %}
:red_circle: CRITICAL ALERT
{% elif event()["severity"] == "HIGH" %}
:large_orange_circle: HIGH ALERT
{% else %}
:large_yellow_circle: {{ event()["severity"] }} ALERT
{% endif %}
```

## Severity Emoji Map

| Severity | Slack Emoji                        | Teams Color |
|----------|------------------------------------|-------------|
| CRITICAL | <code>:red_circle:</code>          | Attention   |
| HIGH     | <code>:large_orange_circle:</code> | Warning     |
| MEDIUM   | <code>:large_yellow_circle:</code> | Accent      |
| LOW      | <code>:large_blue_circle:</code>   | Good        |

## Inline Severity Mapping

```
{{ {"CRITICAL": ":red_circle:", "HIGH": ":large_orange_circle:", "MEDIUM":
":large_yellow_circle:", "LOW": ":large_blue_circle:"}.get(event()
["severity"], ":white_circle:") }}
```

## 7. Complete Alert Workflow Example

A production-ready workflow that sends alerts to Slack, Teams, and email.

## Workflow Configuration

```
name: production-alert-notifications
description: Send alerts to all channels for production problems

trigger:
  type: davis-problem
  config:
    entityTagsMatch: all
    entityTags:
      - key: env
        value: prod

tasks:
  - name: slack_notification
    type: dynatrace.slack:message
    input:
      connection: slack-production
      channel: "#alerts-production"
      message: |
        {{ {"CRITICAL": ":red_circle:", "HIGH": ":large_orange_circle:",
"MEDIUM": ":large_yellow_circle:"}.get(event()["severity"], ":white_circle:")
}} *{{ event()["severity"] }} Problem*

        *{{ event()["title"] }}*

        • *Status:* {{ event()["status"] }}
        • *Started:* {{ event()["start_time"] }}
        • *ID:* {{ event()["display_id"] }}

        &lt;{{ event()["problem_url"] }}|:mag: View Problem&gt;

  - name: teams_notification
    type: dynatrace.msteams:message
    input:
      connection: teams-production
      message: |
        **{{ event()["severity"] }} Problem**

        **{{ event()["title"] }}**

        - **Status:** {{ event()["status"] }}
        - **Started:** {{ event()["start_time"] }}
        - **ID:** {{ event()["display_id"] }}

        [View Problem]({{ event()["problem_url"] }})
```

```

- name: email_notification
  type: dynatrace.email:send
  input:
    to: ["platform-oncall@company.com"]
    subject: "[{{ event()['severity'] }}] [{{ event()['title'] }}"
    body: |
      A [{{ event()["severity"] }}] problem has been detected.

      Problem: [{{ event()["title"] }}]
      Status: [{{ event()["status"] }}]
      Started: [{{ event()["start_time"] }}]

      View: [{{ event()["problem_url"] }}]

```

## Task Execution

By default, tasks execute **in parallel**. All three notifications send simultaneously.

## Monitor Your Notification Workflows

```

// Notification workflow executions
fetch events, from: now() - 24h
| filter event.type == "automation.workflow.execution"
| filter matchesPhrase(workflow.name, "alert") or
  matchesPhrase(workflow.name, "notification")
| fields timestamp,
  workflow.name,
  execution.status,
  execution.duration
| sort timestamp desc
| limit 50

```

```

// Notification task success rates
fetch events, from: now() - 7d
| filter event.type == "automation.task.execution"
| filter matchesPhrase(task.type, "slack") or matchesPhrase(task.type,
  "msteams") or matchesPhrase(task.type, "email")
| summarize
  total = count(),
  succeeded = countIf(task.status == "SUCCEEDED"),
  failed = countIf(task.status == "FAILED"),
  by:{task.type}
| fieldsAdd success_rate = round(100.0 * succeeded / total, decimals: 2)
| sort total desc

```

```
// Failed notification tasks
fetch events, from: now() - 24h
| filter event.type == "automation.task.execution"
| filter task.status == "FAILED"
| filter matchesPhrase(task.type, "slack") or matchesPhrase(task.type,
"msteams") or matchesPhrase(task.type, "email")
| fields timestamp, workflow.name, task.name, task.type, task.error
| sort timestamp desc
| limit 20
```

## Next Steps

---

Now that you can send basic notifications, learn advanced patterns:

### Recommended Path

1. **WFLOW-04: Advanced Notification Routing** - Conditional routing by severity, team, time
2. **WFLOW-05: PagerDuty & ServiceNow** - Incident management integration
3. **WFLOW-06: Custom Notification Templates** - Rich formatting and dynamic content

### Key Takeaways

- **Connections** store credentials securely and are reusable
- **Slack** supports both OAuth apps and webhooks
- **Teams** requires Power Automate or Teams Workflows connector (O365 Incoming Webhooks are deprecated)
- **Email** can use built-in SMTP or custom servers
- **Jinja expressions** enable dynamic message content
- Tasks execute in **parallel** by default

---

## Summary

In this notebook, you learned:

- Notification architecture: triggers → tasks → connections → external services
  - How to set up connections for Slack, Teams, and email
  - Basic and advanced Slack message formatting
  - Microsoft Teams notifications with Adaptive Cards
  - Plain text and HTML email notifications
  - Jinja expressions for dynamic content
  - A complete multi-channel alert workflow
- 

## References

---

- [Workflow Notifications](#)
  - [Slack Actions](#)
  - [Microsoft Teams Actions](#)
  - [Email Actions](#)
  - [Jinja Expressions](#)
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*